

•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- Handling NULL Values with PQL

On this page

Handling NULL Values with PQL

Was this page helpful? 

NULL is a special value that represents the absence of a valid value. Any operation that acts on it as if it was a valid value fails with a `NullPointerException`. Untreated datasets might contain NULL values, and so they are potentially unsafe for any activity within Pulse. This topic describes how to make datasets more robust by handling NULL values using PQL expressions.

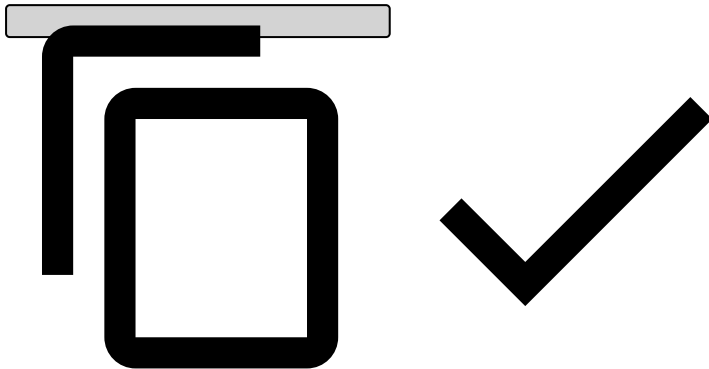
Replacing NULL values

One of the common ways to handle NULLs is to replace them with a value that fits into the data field's type. It's common to pick the substitute value so that it doesn't represent a meaningful value for that data field. For instance, a NULL in the

transaction amount field could be replaced by 0. To accomplish this replacement, you can use the **coalesce operator** (??) in a map operator or metric operator.

Usage of the ?? operator:

```
<good-case-value> ?? <null-case-value>
```



This expression returns `<good-case-value>` , unless `<good-case-value>` is NULL. If `<good-case-value>` is NULL, the expression returns `<null-case-value>` .

Example:

Coalescing aggregations to a value other than 0

Aggregations return 0 instead of a NULL when there are no values to count the aggregation on. For example, when the maximum amount is queried over a time window using a filter that filters out all the records.

In some cases you might want to return something different from 0.

The following example shows how to return -1 in case an aggregation of a metric is called on an empty subset of data:

Nullable types

Untreated input fields can contain NULL values. Hence, PQL treats every input field as a nullable type, meaning a type that can contain NULL values in addition to an original data type.

This fact is marked with a ? in the data type name.

For example, the *Amount* field is not of *double* type, but of *double?*, meaning a type that contains all double values plus the NULL.

When using both nullable and non-nullable types in an [arithmetic operator](#) or in a [function](#) that returns a numeric value, the resulting data type is nullable.